

Nsight Compute

算子性能诊断框架

从“看指标”到“提出假设 - 修改 kernel - 验证”的完整闭环

面向 CUDA GEMM / CUTLASS 学习与算子优化实践

使用方法

不要把这份手册当作指标字典。每次跑 Nsight Compute 时，从第 1 层开始按顺序诊断；只有上一层出现异常，才继续往下追。真正要训练的是“指标 -> 原因 -> 代码 -> 修改”的因果链。

1. 核心目标：不是比较 30 us 和 24 us

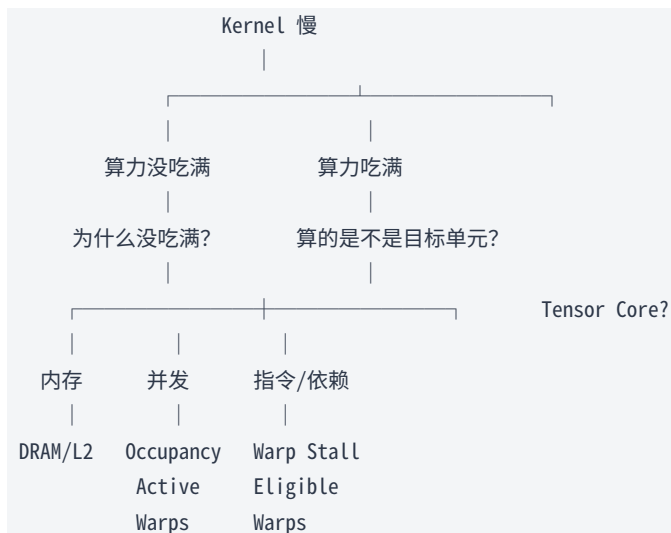
高级算子优化的目标，不只是知道 kernel B 比 kernel A 快，而是能够解释“为什么快、瓶颈在哪里、下一步改什么、修改后指标是否按预期变化”。



最重要的习惯

看到任何一个指标时，不要先问“这个数高不高”，而要问：“它限制了谁？它造成了什么后果？我能从代码层面改什么？”

2. 一张诊断树：先确定“GPU 为什么没吃满”



对 GEMM 来说，建议先只掌握下面这 10 个指标/指标组。它们已经足够覆盖大部分初级到中级问题。

2.1 十个核心指标速查

顺序	指标	核心问题
①	SM Throughput	计算单元忙不忙？
②	Memory Throughput	内存子系统忙不忙？
③	DRAM Throughput / L2 Hit Rate	数据到底来自 DRAM 还是缓存？
④	Tensor Core Utilization	Tensor Core 是否真正持续工作？
⑤	Occupancy	SM 上能驻留多少 warp？
⑥	Registers / Thread	寄存器是否限制驻留？
⑦	Shared Memory / Block	共享内存是否限制驻留？
⑧	Active Warps	实际驻留的 warp 多不多？
⑨	Eligible Warps	真正 ready、可发射的 warp 多不多？
⑩	Warp Stall Reasons	warp 为什么不能发射？

3. 第一层：SM Throughput 与 Memory Throughput

打开 NCU 后，第一问不是“哪个 stall 最大”，而是：GPU 到底是在忙着算，还是忙着搬数据？

3.1 三种最常见组合

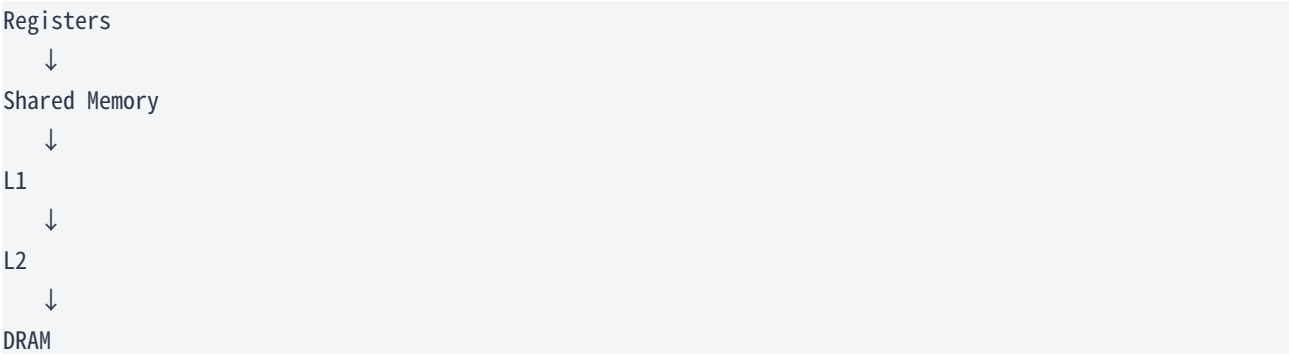
情况 A	
SM Throughput	85%
Memory Throughput	30%
→ 更像 Compute Bound	
情况 B	
SM Throughput	35%
Memory Throughput	90%
→ 更像 Memory Bound	
情况 C	
SM Throughput	30%
Memory Throughput	25%
→ 两边都没吃满，继续查 latency / dependency / scheduling / occupancy	

记忆规则

SM 高 + Memory 低：Compute-bound 倾向；SM 低 + Memory 高：Memory-bound 倾向；两者都低：优先怀疑 warp 没有 ready，而不是简单归类为“带宽不够”。

4. Memory 高时：继续拆 DRAM 与 L2

Memory Throughput 高并不等于 DRAM 一定满。GPU 的数据路径是一个层次结构：



4.1 典型：DRAM 压力真的很大

Memory Throughput	90%
DRAM Throughput	88%
L2 Hit Rate	20%

- 大量请求穿过 L2 去了显存
- 关注数据复用、global load 次数、coalescing、shared-memory tiling

4.2 典型：Memory 忙，但大量命中 L2

Memory Throughput	85%
DRAM Throughput	30%
L2 Hit Rate	90%

- 不是简单的 DRAM bandwidth bottleneck
- 继续排查 cache / load-store pipeline / shared memory / issue

5. Tensor Core：写了 MMA 不等于 Tensor Core 吃满

在 WMMA / CUTLASS GEMM 中，必须把“调用了 Tensor Core 指令”和“Tensor Core 持续处于高利用率”区分开。

SM Throughput	80%
Tensor Core Utilization	20%

可能意味着：

SM 在忙 load/store、地址计算、转换、shared-memory 操作、同步等，
而 Tensor Core 反而经常在等数据。

理想流水：

load → mma → load → mma → load → mma

糟糕流水：

load → wait → wait → mma → load → wait → wait → mma

因此，当 Tensor Core utilization 低时，下一问应该是：“为什么 MMA 指令不能持续获得数据并被发射？”

6. Active Warps 与 Eligible Warps：理解调度的关键

这两个指标是从“资源占用”过渡到“GPU 为什么空转”的桥梁。

- Active Warps：已经驻留在 SM 上的 warp。
- Eligible Warps：当前已经 ready，可以被 scheduler 选中发射指令的 warp。

```
Warp 0: 等 global memory
Warp 1: 等 shared memory
Warp 2: 等 dependency
Warp 3: ready
Warp 4: 等 barrier
Warp 5: ready
```

Active: 很多

Eligible: 只有少数

关键认知

Occupancy 高 $\neq$ GPU 一定利用率高。你可以有很多驻留 warp，但如果它们都在等数据，Eligible Warps 仍然很低。

7. 什么时候看 Warp Stall Reasons

不要一上来就看一长串 Stall Reasons。正确顺序是：

```
SM Throughput 低
Memory Throughput 也不高
Active Warps 不低
Eligible Warps 很低
    ↓
现在才问：warp 为什么不能执行？
    ↓
Warp Stall Reasons
```

7.1 现阶段优先掌握的 5 类 Stall

- Long Scoreboard：通常表示等待较长延迟的数据依赖，常见于 global/L2 load。
- Short Scoreboard：更偏短延迟依赖，例如 shared-memory 相关依赖。
- Barrier：warp 在等待同步点，例如 `__syncthreads()`。
- Math Pipe Throttle：某类数学执行管线繁忙；如果目标就是把该单元压满，它未必是坏事。
- Not Selected：warp 已经 ready，但 scheduler 选择了其他 warp；很多时候说明 ready warp 很充足。

8. Stall 与代码修改如何对应

Long Scoreboard ↑

→ 数据返回慢 / latency hiding 不足

→ 尝试：数据复用、prefetch、更多并发 load、software pipeline、double buffering

Short Scoreboard ↑

→ 短依赖链太紧

→ 尝试：register tiling、ILP、重排 load/compute

Barrier ↑

→ 同步等待明显

→ 尝试：减少同步、async copy、pipeline、双缓冲

Tensor Core 低 + wait 多

→ MMA 供数不连续

→ 尝试：完善 global→shared→register→MMA 流水

9. Occupancy：不是越高越好

Occupancy 的意义是“SM 可以驻留多少 warp”，它主要受 Registers、Shared Memory、Threads/Block 等资源限制。

Registers / Thread ↑

↓

每个 Block 占用 Registers ↑

↓

一个 SM 可驻留 Block 数 ↓

↓

Occupancy ↓

但降低 occupancy 有时是值得的：例如 register tile 增大后数据复用和 ILP 提升，性能可能仍然上升。真正要看的是 occupancy 的变化是否导致 Active/Eligible Warps 不足。

正确问法

不要问“Occupancy 只有 50%，是不是太低？”；要问“50% 是否已经导致 Eligible Warps 不足，从而无法隐藏延迟？”

10. Registers / Thread: 把数字串成因果链

TM/TN 增大
↓
每线程 accumulator 增多
↓
Registers / Thread
64 → 120
↓
Occupancy
75% → 37%
↓
Active Warps ↓
↓
Latency Hiding ↓
↓
Long Scoreboard ↑
↓
SM Throughput ↓
↓
Kernel 24 us → 29 us

这才是性能分析中的“解释”。单独写一句“register = 120”几乎没有价值。

11. Shared Memory / Block: 同样要看它限制了什么

假设:
SM 可用 Shared Memory = 64 KB
Kernel Shared Memory = 48 KB / Block

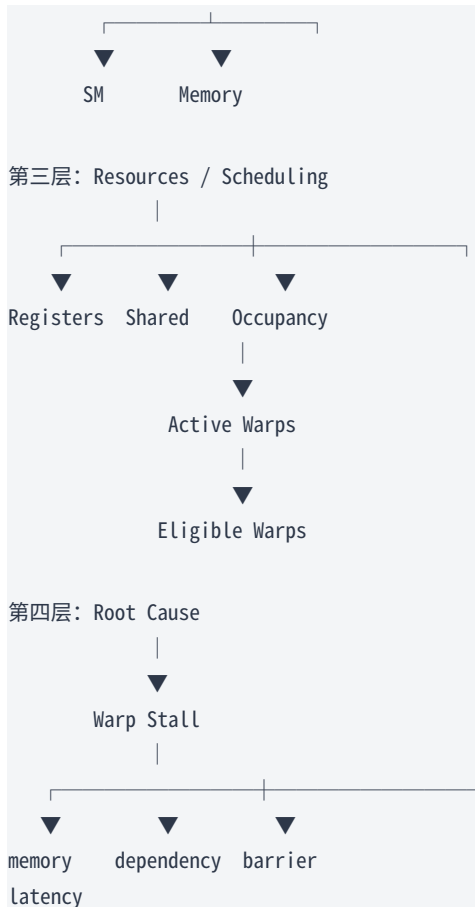
→ 一个 SM 可能只能放 1 个 Block
→ 如果 Block 只有 128 threads = 4 warps
→ Active Warps 很少
→ latency hiding 能力可能不足

因此，扩大 tile 虽然提高数据复用，但也可能通过 shared-memory 占用降低并发度。

12. 四层模型：以后按这个顺序看 NCU

第一层: Performance
|
▼
Latency

第二层: Hardware Utilization
|



诊断方向

从上往下诊断；从下往上解释性能。

13. 完整案例 A: Naive GEMM -> Shared Memory Tiling

Before	
Latency	120 us
SM Throughput	28%
Memory Throughput	82%
DRAM Throughput	78%
L2 Hit Rate	25%

第一层判断：Memory 很高、SM 很低，memory-bound 倾向。DRAM 高且 L2 hit 低，说明大量访问真正去了显存。

```

naive GEMM:
for (int k = 0; k < K; ++k) {
    sum += A[row*K+k] * B[k*N+col];
}
  
```

问题：A/B 数据被不同线程反复从 global memory 读取。

Hypothesis: 主要瓶颈是 global-memory 重复访问和数据复用不足。

修改: 使用 shared-memory tiling, 让一个 tile 的 A/B 在 block 内被多个线程复用。

After	
Latency	55 us
SM Throughput	65%
Memory Throughput	60%
DRAM Throughput	40%
解释: DRAM 压力下降、SM 利用率上升, 指标变化与 hypothesis 一致。	

14. 完整案例 B：Tile 变大，反而更慢

修改：Tile 32×32 → 64×64

结果：

Latency 55 us → 68 us
Registers / Thread 40 → 90
Shared Memory / Block 16 KB → 48 KB
Occupancy 75% → 25%
Active Warps ↓
Eligible Warps ↓
Long Scoreboard ↑

Tile ↑
↓
理论 Data Reuse ↑

但同时：
Registers ↑ + Shared Memory ↑
↓
Resident Blocks ↓
↓
Occupancy / Active Warps ↓
↓
Latency Hiding ↓
↓
Eligible Warps 不足
↓
Pipeline 空转
↓
性能反而下降

这就是算子优化的核心

一个参数通常同时改善一件事、恶化另一件事。CUTLASS 参数空间的本质，就是在 Data Reuse、Arithmetic Intensity、Registers、Shared Memory、Occupancy、Scheduling 之间寻找平衡。

15. NCU 实战填写模板

Kernel	_____
Latency	_____ us
SM Throughput	_____ %

Memory Throughput	_____ %
初步判断	Compute / Memory / Latency / Scheduling ?
DRAM Throughput	_____ %
L2 Hit Rate	_____ %
Tensor Core Utilization	_____ %
Registers / Thread	_____
Shared Memory / Block	_____
Occupancy	_____ %
Active Warps	_____
Eligible Warps	_____
Top Stall 1	_____
Top Stall 2	_____
Top Stall 3	_____
Hypothesis	我认为瓶颈是： _____
证据	Metric A + Metric B + Metric C: _____
准备修改	_____
After Latency	_____ us
关键指标变化	_____
结论	Hypothesis 是否被验证? _____

16. 最终速查：真正需要记住的顺序

```
graph TD; A[Throughput] --> B[Memory / Compute]; B --> C[DRAM / L2 / Tensor Core]; C --> D[Registers / Shared / Occupancy]; D --> E[Active Warps]; E --> F[Eligible Warps]; F --> G[Warp Stall]; G --> H[回到代码]; H --> I[提出 Hypothesis]; I --> J[修改 + Benchmark + 再 Profile];
```

Throughput
↓
Memory / Compute
↓
DRAM / L2 / Tensor Core
↓
Registers / Shared / Occupancy
↓
Active Warps
↓
Eligible Warps
↓
Warp Stall
↓
回到代码
↓
提出 Hypothesis
↓
修改 + Benchmark + 再 Profile

当你能稳定走完这条链路时，Nsight Compute 就不再是一堆互相独立的百分比，而会变成一套可以指导代码修改的诊断系统。